

std::ranges::contains

Document #: P2302R3
Date: 2021-02-10
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply-to: Christopher Di Bella
<cjdb.ns@gmail.com>

Contents

- 1 Abstract
- 2 Revision history
 - 2.1 R2
 - 2.2 R1
 - 2.3 R0
- 3 Motivation
- 4 Design
 - 4.1 API
 - 4.2 Naming
 - 4.3 (haystack, needle) or (needle, haystack)?
- 5 Proposed wording
 - 5.1 Feature-test macro
- 6 Implementation experience
- 7 Acknowledgements
- 8 References

§ 1 Abstract

P2302 proposes two algorithms: one that checks whether or not a range contains an element, and one that checks whether or not a range contains a subrange.

§ 2 Revision history

§ 2.1 R2

- Renames the subrange overload to `contains_subrange`.
- Adds bit about implementation experience.

§ 2.2 R1

- Removes request to have `basic_string[_view]::contains` struck from the C++23 working paper.
 - Adjusts motivation to match.
- Adds section on naming.

§ 2.3 R0

- Initial revision

§ 3 Motivation

At present, it isn't immediately possible to query whether or not any range—other than a C++23 standard string type— contains an element. [\[P1679R3\]](#) links `basic_string[_view]::contains` with `basic_string[_view]::starts_with` and `basic_string[_view]::ends_with`. Although C++ has done without a dedicated `contains` algorithm for over twenty years, it'd be unfortunate if we only strings had a `contains` algorithm.

This functionality is useful for any input range, when the user wants to know whether or not their range simply contains a value. Up until C++20, we've had to write `std::find(r, value) != std::end(r)` to determine if a single value is inside a range, and to check if a range contains a subrange of interest, we use not `std::search(haystack, needle).empty()`. While this is accurate, it isn't necessarily convenient, and it hardly expresses intent (especially in the latter case). Being able to say `std::contains(r, value)` addresses both of these points. Further evidence for the existence of a `contains` algorithm is that the STL gave C++98 an obscurely-named `contains` algorithm called `binary_search`, and `any_of` is “contains with a predicate”, showing that there's prior art for this `contains` to be an algorithm.

§ 4 Design

The proposed usage looks like this:

```
if (std::ranges::contains(haystack, 'o')) {  
    // ...  
}  
  
if (std::ranges::contains(haystack.begin(), haystack.end(), 'c')) {  
    // ...  
}
```

```

}

if (std::ranges::contains_subrange(haystack.begin(), haystack.end(),
    long_needle.begin(), long_needle.end())) {
    // ...
}

if (std::ranges::contains_subrange(haystack, long_needle)) {
    // ...
}

if (std::ranges::contains_subrange(haystack.begin(), haystack.end(),
    long_needle.begin(), long_needle.end(), bind_back(std::modulo(),
    4))) {
    // ...
}

if (std::ranges::contains_subrange(haystack, long_needle,
    bind_back(std::modulo(), 4))) {
    // ...
}

```

Due to length, the comparison tables below only consider the range-based overloads.

Before	After
<pre> namespace stdr = std::ranges; stdr::find(haystack.begin(), haystack.end(), 'o') != haystack.end() stdr::find(haystack, 'o') != stdr::end(haystack) not stdr::search(haystack, long_needle).empty() not stdr::search(haystack, long_needle, bind_back(std::modulo(), 4)).empty() </pre>	<pre> namespace stdr = std::ranges; stdr::contains(haystack.begin(), haystack.end()) stdr::contains(haystack, 'o') stdr::contains_subrange(haystack, long_needle) stdr::contains_subrange(haystack, long_needle, bind_back(std::modulo(), 4)) </pre>

§ 4.1 API

As this is an algorithm, the author suggests this be placed in namespace `std::ranges`, where new algorithms are currently destined to go. As usual with new algorithms, these ones will also have projections.

The author notes that for an unsorted sequence, a `contains` algorithm is simply one of the following:

```

auto const haystack = "the quick brown fox jumps over the lazy dog"sv;

stdr::find(haystack, 'o') != haystack.end(); // haystack.contains('o')
not stdr::search(haystack, "red"sv).empty(); // haystack.contains("red"sv)

```

```
// See https://godbolt.org/z/vTendW
```

As such, `contains` is a wrapper around these two algorithms:

```
namespace std {
    template<input_iterator I, sentinel_for<I> S, class T, class Proj =
        identity>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>,
        const T*>
    constexpr bool ranges::contains(I first, S last, const T& value, Proj
        proj = {});

    template<input_range R, class T, class Proj = identity>
    requires indirect_binary_predicate<ranges::equal_to,
        projected<iterator_t<R>, Proj>, const T*>
    constexpr bool ranges::contains(R&& r, const T& value, Proj proj = {});

    template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
        sentinel_for<I2> S2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
    constexpr bool
        ranges::contains_subrange(I1 first1, S1 last1, I2 first2, S2 last2,
            Pred pred = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});

    template<forward_range R1, forward_range R2, class Pred =
        ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
    constexpr bool
        ranges::contains_subrange(R1&& r1, R2&& r2, Pred pred = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});
}
```

§ 4.2 Naming

The minutes from P2302R0's discussion note that it may be more appropriate to name these functions `contains_element` and `contains_range`, since it could be ambiguous as to whether or not the range-based algorithms are performing a find or a search. Worse, a programmer might intend to use the element `contains` while searching a range of ranges, which without care, will accidentally call the wrong overload. This is a good point, and the names should probably be different.

This paper proposes `contains` instead of `contains_element`, since that is a very common, and much complained-about missing feature in C++. It also elects to rename the suggested `contains_range` as `contains_subrange`, because the author feels it's a more appropriate name: if the range is found, then it's a subrange of the haystack.

§ 4.3 (haystack, needle) or (needle, haystack)?

This issue was addressed in [P1659R3], but it bears repeating.

Concerns were outlined in all of N3609, P0457, P1679 about the ambiguity of whether we are performing `starts_with(haystack, needle)` or `starts_with(needle, haystack)`. There is prior art in the algorithms library that makes the first range the subject of the operation: `mismatch`, `equal`, `search`, `find_first_of`, and `lexicographical_compare` all take the form `algorithm(haystack, needle)`, so the author remains unconvinced about the ambiguity.

LEWG approved P1659 in the Cologne 2019 meeting: meaning that this working group agrees that there isn't any ambiguity, or that LEWG is inconsistent in what it agrees upon.

§ 5 Proposed wording

Add the following text to 25.4 [algorithm.syn]:

```
// [alg.none.of], none of
// ...

// [alg.contains], contains
template<input_iterator I, sentinel_for<I> S, class T, class Proj =
    identity>
    requires indirect_binary_predicate<ranges::equal_to, projected<I,
        Proj>, const T*>
    constexpr bool ranges::contains(I first, S last, const T& value, Proj
        proj = {});
template<input_range R, class T, class Proj = identity>
    requires indirect_binary_predicate<ranges::equal_to,
        projected<iterator_t<R>, Proj>, const T*>
    constexpr bool ranges::contains(R&& r, const T& value, Proj proj =
        {});

template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
    sentinel_for<I2> S2, class Pred = ranges::equal_to,
    class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
    constexpr bool ranges::contains_subrange(I1 first1, S1 last1, I2
        first2, S2 last2,
        Pred pred = {}, Proj1 proj1 =
            {}, Proj2 proj2 = {});
template<forward_range R1, forward_range R2, class Pred =
    ranges::equal_to,
    class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
    constexpr bool ranges::contains_subrange(R1&& r1, R2&& r2, Pred pred =
        {},
        Proj1 proj1 = {}, Proj2 proj2
            = {});

// [alg.foreach], for each
```

Add the following to 25.6 [\[alg.nonmodifying\]](#):

```
template<input_iterator I, sentinel_for<I> S, class T, class Proj =
    identity>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>,
        const T*>
    constexpr bool ranges::contains(I first, S last, const T& value, Proj
        proj = {});

template<input_range R, class T, class Proj = identity>
    requires indirect_binary_predicate<ranges::equal_to,
        projected<iterator_t<R>, Proj>, const T*>
    constexpr bool ranges::contains(R&& r, const T& value, Proj proj = {});
```

Returns: `ranges::find(std::move(first), last, value, proj) != last;`

```
template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
    sentinel_for<I2> S2, class Pred = ranges::equal_to,
    class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
    constexpr bool ranges::contains_subrange(I1 first1, S1 last1, I2 first2,
        S2 last2,
        Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<forward_range R1, forward_range R2, class Pred =
    ranges::equal_to,
    class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
    constexpr bool ranges::contains_subrange(R1&& r1, R2&& r2, Pred pred =
        {},
        Proj1 proj1 = {}, Proj2 proj2 = {});
```

Returns: `first2 == last2 || !ranges::search(first1, last1, first2, last2, pred, proj1, proj2).empty();`

§ 5.1 Feature-test macro

Add the following macro definition to 17.3.2 [\[version.syn\]](#), header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_contains 20XXXXL // also in <algorithm>
```

§ 6 Implementation experience

This has been implemented in range-v3 and is available as a [Pull Request](#).

§ 7 Acknowledgements

The author would like to thank Corentin Jabot for reviewing this proposal.

§ 8 References

[P1659R3] Christopher Di Bella. 2021-02-19. `starts_with` and `ends_with`.
<https://wg21.link/p1659r3>

[P1679R3] Wim Leflere, Paul Fee. 2020-07-22. `String Contains` function.
<https://wg21.link/p1679r3>